

STARTING A PYTHON RESEARCH PROJECT IN 2026

Moacir P. de Sá Pereira

Research Data Librarian | **Columbia University Libraries**

mpd2149@columbia.edu | [he/him/his](#)

NY, NY, 2026

<https://columbia-data.club/iay02j>

Opening Links:

- Install:
 - Git:
 - Mac: `xcode-select --install` in Terminal
 - Windows: **Git for Windows**
- Create an Account on:
 - **GitHub** (Requires email verification)

About Me

- Research Data Librarian at **Columbia University Libraries**
- Taught English at NYU before
- Occasional scholar of literature and space
- Contact me and my colleagues at `data@library.columbia.edu`

Data Club

linktr.ee/columbialibdata

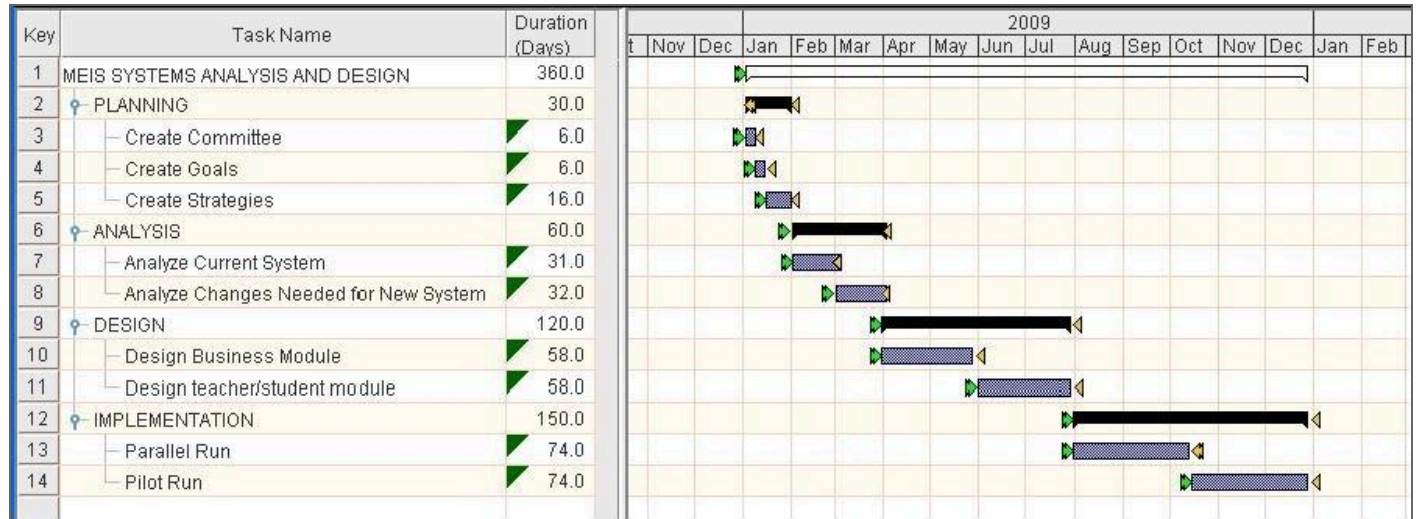
- › Jan 29 (Thu): **Working with Large Datasets in R**
- › Feb 4 (Wed): **Open Source GIS 1: Data Structures, Manipulation, Etc.**
- › Feb 5 (Thu): **Working with Repetitive Data Tasks in R**
- › Feb 11 (Wed): **Open Source GIS 2: Intro to Visualization and Cartography.**
- › Feb 12 (Thu): **Move beyond Jupyter Notebooks with Marimo**

What follows is not the only way to proceed.
It's not necessarily the best way.
But it's an OK way for an illustration.
And it's a workable foundation.

Outline

- Projects in general
- Projects in Python with **uv**
- Git with **uv**
- Google Cloud Data Storage (?)

Your Research Is Not a *Document*. It Is a *Project*.



“Project Management Simplified” (**LinkedIn Course**)

- Know your constraints
- Break your project apart
- Find your blockers

Projects Go in Folders

- Many resources
- Many dependencies
- Many versions

uv

- Replaces `pip`, `virtualenv`, `pyenv`, and `poetry`.
- Blazingly fast
- Install once and link forever dependency management (more `pnpm` than `npm`)

Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

In Powershell:

```
powershell -ExecutionPolicy ByPass -c "irm https://
```

Start a Project

```
uv init my-project
```

This creates a folder `my-project` and puts five files in it:

- `README.md`, which introduces users to your project
- `main.py`, which is the main point of entry into your project (for code)
- `pyproject.toml`, which is a **TOML**-formatted config file that describes the Python infrastructure, including dependencies
- `.python-version`, which lists the canonical version of Python for the project
- `.gitignore`, which tells Git which files to ignore

Dependencies

Dependencies are the various software libraries that you install in addition to the regular vanilla Python you use in your project. Lines of code like `import polars as pl` indicate that your project relies on the **Polars** data wrangling library. Let's add **Ruff**, a Python linter and formatter, to our project:

```
uv add ruff
uv run ruff check
uv run main.py
```

Venv

uv gives you a virtual environment for free. That's a vital part of its flexibility.

All of the dependencies are flagged in the virtual environment, but they are not installed in the project. Rather, they are installed for the user once in a global cache and then linked to the project. This means multiple projects can reference the same library in separate virtual environments, but it is only installed once.

Bringing uv to an Existing Python Project

If you already have a Python project that uses `pip` and has a `requirements.txt`, the upgrade path is simple:

```
uv init
uv add -r requirements.txt
uv sync
```

This incorporates all of the libraries in the `requirements.txt` file into a `pyproject.toml` file and makes everything `uv`-native. Now you can remove `requirements.txt`.

Upgrading from poetry is a little more involved, but you can use `migrate-to-uv`.

Using Different Versions of Python

```
uv python list  
uv python install 3.10 3.11 3.12
```

Sometimes different projects require different versions not just of dependencies but also of Python itself. As expected, **uv** can help with that.

Now, Git.

Git Is:

- “Free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.”
- “Easy to learn.”
- A busybody keeping track of all the changes to all your files and never forgetting them.

Git Is:

1. Part of a backup solution
2. An intention tracker/writing journal
3. A declutterer
4. A multi-verse generator
5. A collaboration engine

The Four Main Steps to Git

1. Save - Not even part of Git.
2. Stage - Also known as `git add`.
3. Commit - Show your *intention*.
4. Push - Sync up and backup.



Save your changes



Stage them with Git



Commit them with Git

Push them to the cloud!



Icons by Font Awesome. **License**.

You're not working on a *document*.
You're working on a *project*.

I Like Lazygit

Lazygit is a great tool that presents Git's complexities in a somewhat manageable way. The easiest way to install it (on Macs) is with **Homebrew**:

```
brew install lazygit
```

(We could've installed `uv` with Homebrew, too!)

KEEP TRACK OF WHAT YOU DID

With Git or not...

Google Cloud Storage

Cloud Storage, which is a part of the **Google Cloud Platform**, gives you access to storage buckets online. They are not free, but they are cheap, and Columbia researchers can start their GCP careers with a stack of free credits.

First we create the bucket:

- › Go to the **GCP console**
- › Create a new project or check you're in the right one
- › Click “Create Storage Bucket”
- › Select all the defaults
- › Upload something

Manage Access

- › Go to the **GCP console**
- › Scroll to IAM & Admin
- › Pick “Service Accounts” on the left
- › Click “Create Service Account”
- › Fill in info and click “Create and continue”
- › Give it “Storage Object Viewer” permissions, and finish up.
- › Select the account, go to Keys, and create a private JSON key.
- › Save the file to your project folder.
- › Add the file name to your .gitignore!

Access from Python

```
uv add polars google-cloud-storage
```

In `main.py`:

```
from google.cloud import storage
from google.oauth2 import service_account
import polars as pl
import io

KEY_PATH = "data-club.json"
BUCKET = "data-club-foo-1"
BLOB = "data.csv"

def main():
    creds = service_account.Credentials.from_servi
```

DISCO! 🕺

There are tidier ways to wire up a Python script to Cloud Storage, but this one relies on the least amount of dependencies.

Next Steps

- Read Kieran Healy's ***The Plain Person's Guide to Plain Text Social Sciences*** for tips, inspiration, alternatives, etc.
- Google around, talk to me...

KEEP TRACK OF WHAT YOU DID

With Git or not...

Thanks!

Speaker notes